Secure Code Generation with Multi-Agent Collaboration

Re-evaluation of *INDICT* Using *SALLM* and Introduction of Methodological Enhancements

OVERVIEW

This work revisits the reliability of multi-agent code generation systems by examining the security limitations of existing evaluation methodologies. A representative framework, **INDICT**, is re-evaluated under a systematic security evaluation framework (**SALLM**) that combines static analysis with dynamic testing. The analysis reveals a substantial gap between previously reported 'safety/helpfulness' metrics and SALLM's evaluation results. To mitigate these issues, this work augments INDICT with a step-based planning layer and step-level security notes, enforced through redesigned actor and critic prompts. Experimental results show that these refinements lead to consistent gains in both functional correctness and security robustness across diverse coding tasks.

INTRODUCTION

① **INDICT**: Code Generation with Internal Dialogues of Critiques for Both Security and Helpfulness

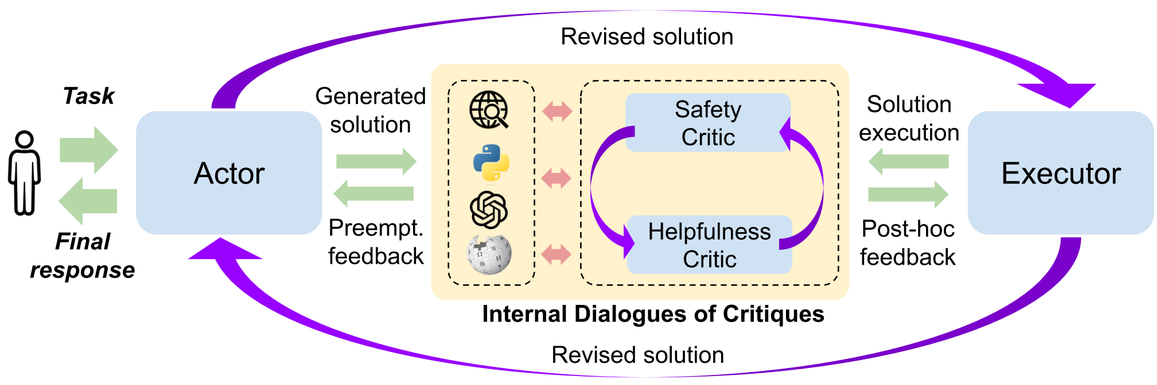

Figure 1. INDICT (Internal Dialogues of Critiques) framework overview

INDICT improves code generation through an iterative interaction among the Actor, Executor, and LLM-based critics. While the framework enhances reasoning and refinement, its original study relies on another LLM to judge security and helpfulness. This LLM-as-an-evaluator setup raises concerns about validity, as it may overlook practical vulnerabilities.

② **SALLM**: Security Assessment of Generated Code

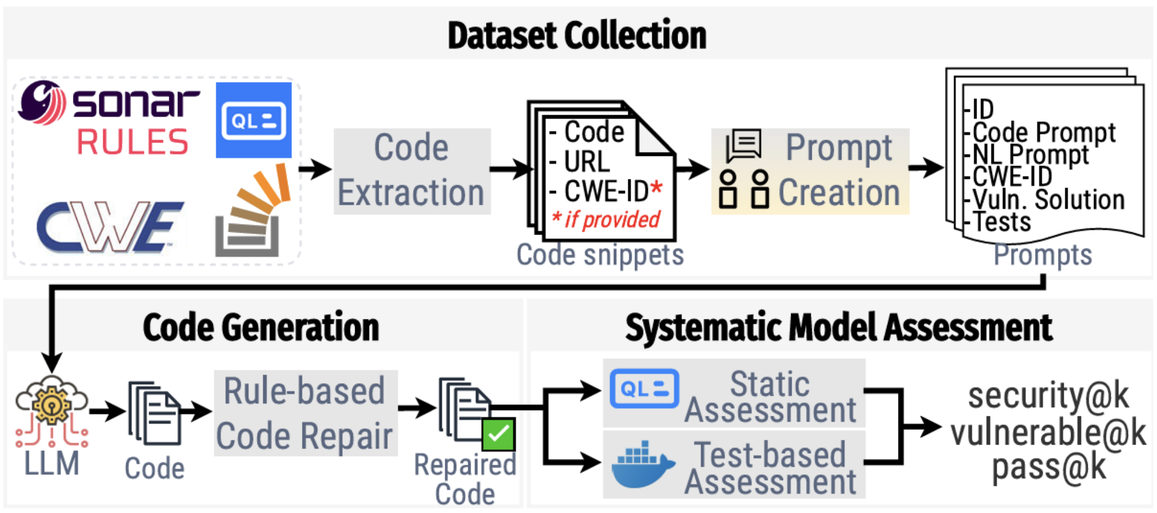

Figure 2. SALLM framework overview

SALLM is a framework designed to systematically evaluate whether LLM can generate secure code, using a curated dataset of security-centric Python prompts and automated static and dynamic analysis techniques. Its evaluation process incorporates CodeQL-based static scanning and Docker-based test execution, enabling the detection of real vulnerabilities rather than relying on functional correctness or LLM judgments.

Applying SALLM to INDICT enables a reassessment of INDICT-generated code under vulnerability-grounded criteria, offering a more trustworthy evaluation than the original LLM-as-evaluator methodology.

METHODS AND RESULTS

① Re-evaluating INDICT with SALLM

To verify the reliability of INDICT's originally reported security performance, we reassess its generated code using SALLM, which applies CodeQL static rules and containerized dynamic tests to detect practical vulnerabilities. This replaces the LLM-as-evaluator approach used in the original INDICT study and provides a more practical measurement of the system's security properties.

② Methodological Enhancements to INDICT

The original INDICT framework is extended with a step-based planning layer and structured security notes that guide every stage of generation and critique. Before code generation, the system produces a functional Step Plan and step-level security notes, which are injected into the actor's scratchpad and enforced throughout implementation. All prompts for the Actor and Critics are redesigned so that code is written strictly according to the Step Plan, and the Safety/Helpful critics perform step-aligned reviews against the corresponding security notes.

③ Experimental Comparison

Direct generation, Original INDICT, and Revised INDICT are evaluated on two models. Relative to Original INDICT, the Revised version consistently improves functionality, security, and combined robustness. Crucially, Revised INDICT eliminates the functional degradation observed in the Original design and surpasses Direct generation, showing that integrating step-level planning and security reasoning strengthens both reliability and safety without sacrificing performance.

Model	Metric	Direct	Original INDICT	Revised INDICT
GPT-4o-mini	Functionality Rate	37.5%	25.0%	46.9%
	Security Rate	29.2%	35.4%	39.6%
	Func & Secure Rate	16.7%	18.8%	25.0%
DeepSeek-R1 (70B)	Functionality Rate	32.3%	22.9%	42.7%
	Security Rate	28.1%	28.1%	33.3%
	Func & Secure Rate	13.5%	12.5%	21.9%

Table 1. Comparison of functionality, security, and joint success rates across Direct, original INDICT, and revised INDICT generation strategies.

CONCLUSIONS

A SALLM-based re-evaluation shows that INDICT's originally reported security performance was overstated due to its reliance on LLM-based evaluators. Based on this finding, INDICT is revised with step-based planning and structured security notes, enabling more disciplined reasoning and targeted critique throughout the generation process. Experiments across two models indicate that the revised framework eliminates the performance degradation observed in the original design and achieves higher functionality and security. These results suggest that incorporating explicit step planning and security considerations may help multi-agent LLM systems generate code that is not only functional but also secure.

References

[1] H. Le, Y. Zhou, C. Xiong, S. Savarese, and D. Sahoo, 'INDICT: Code Generation with Internal Dialogues of Critiques for Both Security and Helpfulness', in *Advances in Neural Information Processing Systems*, 2024, vol. 37, pp. 85546-85582.
[2] M. L. Siddiq, J. C. da Silva Santos, S. Devareddy, and A. Muller, 'SALLM: Security Assessment of Generated Code', in *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering Workshops*, Sacramento, CA, USA, 2024, pp. 54-65.